

Reviewer Pack — Newton Polytope Vertex Count Bounds SOS Rank for CSP Refutat...

Entry 0ccae330ab62 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 09:51:07 UTC

Newton Polytope Vertex Count Bounds SOS Rank for CSP Refutations

Entry ID: 0ccae330ab62 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 09:51:07 UTC

1. Conjecture

Field A (mathematical branch): REAL_ALGEBRAIC_GEOMETRY **Field B** (complexity object): CSP_LOWER_BOUNDS

Statement:

For a system of polynomial equations defining a CSP, the SOS rank required to refute the CSP is at least the number of vertices of the Newton polytope of the system. Equivalently, the minimal SOS rank needed to certify infeasibility of a CSP instance is lower-bounded by the vertex count of its defining polynomial's Newton polytope.

Rationale (proposer's reasoning):

The Newton polytope encodes the monomial structure of the polynomial constraints, and its vertex count reflects the combinatorial complexity of the feasible region. This geometric invariant may directly influence the SOS hierarchy's ability to find refutations, as vertices correspond to extremal points that require higher rank certificates for non-negativity.

Taxonomy category: SOS_HIERARCHY (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): e658d88024368be6

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    # Generate a random polynomial CSP instance with n variables
    n = random.randint(5, 40)
    coefficients = [[random.uniform(-1, 1) for _ in range(n+1)] for _ in range(n)]
    equations = [sum(coeff * x**i for i, coeff in enumerate(row)) for row in coefficients]

    # Compute the Newton polytope using a convex hull algorithm
    points = [(i, j) for i in range(n+1) for j in range(n+1) if all(coefficients[i][j] != 0)]
    vertices = convex_hull(points)

    # Count the number of vertices
    vertex_count = len(vertices)

    # Compute the minimal SOS rank required for refutation via semidefinite programming
    sos_rank = min_sos_rank(equations, n)

    # Compare vertex counts with SOS rank values
    conjecture_holds = vertex_count <= sos_rank
    counterexample = "" if conjecture_holds else f"Vertex count {vertex_count} > SOS rank {sos_rank}"

    return {
        "metric_name": "SOS Rank",
        "metric_value": sos_rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }

def convex_hull(points):
    # Implement a simple convex hull algorithm (e.g., Graham's scan)
    if len(points) < 3:
        return points
```

```

def orientation(p, q, r):
    val = (q[1] - p[1]) * (r[0] - q[0]) - (q[0] - p[0]) * (r[1] - q[1])
    if val == 0:
        return 0
    elif val > 0:
        return 1
    else:
        return 2

def compare(p, q):
    o = orientation(points[0], p, q)
    if o == 0:
        return (p[0]**2 + p[1]**2) - (q[0]**2 + q[1]**2)
    elif o == 1:
        return -1
    else:
        return 1

points.sort(key=lambda point: (point[0], point[1]))
lower = []
for point in points:
    while len(lower) >= 2 and orientation(lower[-2], lower[-1], point) != 2:
        lower.pop()
    lower.append(point)

upper = []
for point in reversed(points):
    while len(upper) >= 2 and orientation(upper[-2], upper[-1], point) != 2:
        upper.pop()
    upper.append(point)

hull = lower[:-1] + upper[::-1]
return hull

def min_sos_rank(equations, n):
    # Implement a simple semidefinite programming solver to find the minimal SOS rank
    # This is a placeholder and should be replaced with an actual implementation
    return n # Placeholder value for demonstration purposes

if __name__ == "__main__":
    import sys
    seeds = [int(arg) for arg in sys.argv[1:]] or [2**i - 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(res["metric_value"] for res in results) / len(results)
    std_value = math.sqrt(sum((res["metric_value"] - mean_value)**2 for res in results) / len(results))
    support_fraction = sum(1 for res in results if res["conjecture_holds"]) / len(results)

    if all(res["conjecture_holds"] for res in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")

```

```

elif support_fraction >= 0.8:
    print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
else:
    first_failing_seed = next(res["seed"] for res in results if not res["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_daf81ef3.py", line 98, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_daf81ef3.py", line 24, in run_trial
    equations = [sum(coeff * x**i for i, coeff in enumerate(row)) for row in coefficients]
                  ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_daf81ef3.py", line 24, in <genexpr>
    equations = [sum(coeff * x**i for i, coeff in enumerate(row)) for row in coefficients]
                  ^
NameError: name 'x' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'x' before producing results | next: Fix test code to define variables properly and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	117002
2	propose	ollama_remote	qwen3:8b	0	0	156859
3	preregistration	ollama_remote	qwen3:8b	0	0	27429
4	novelty	ollama_remote	qwen3:8b	0	0	24111
5	novelty	ollama_remote	qwen3:8b	0	0	17272
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10268
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10509
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8980
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11268

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
10	judge	ollama_remote	qwen3:8b	0	0	20459

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 404156 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/0ccae330ab62.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/0ccae330ab62.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/0ccae330ab62.tar.gz (if generated)